

Price Drop Tracker & Alert API

Project Roadmap & Tech Stack

Languages & Technologies Used

Layer	Language / Tool	Purpose
Backend logic	Python	Scraping, business rules, alert logic
API framework	FastAPI (Python)	REST endpoints, request/response validation
Database	SQL (PostgreSQL)	Storing products, price history, queries
Scheduling	Python (APScheduler)	Running scrapes automatically on a timer
Frontend	HTML, CSS, JavaScript	Dashboard UI, forms, fetch calls to the API
Charts	JavaScript (Chart.js)	Price trend visualization
Auth	Python (JWT via FastAPI)	Login, per-user data protection
Deployment config	YAML / Shell	Render/Railway deployment settings

Python is the core language (backend, scraping, scheduling, auth), SQL handles all data modeling and queries, and HTML/CSS/JS power the frontend.

4-Week Build Roadmap

1. Week 1 — Foundations & setup

Learn/refresh FastAPI basics (routes, Pydantic models), SQLAlchemy or SQLModel, and PostgreSQL fundamentals (CREATE TABLE, JOIN, GROUP BY). Set up your dev environment (Python venv, Postgres running locally or via Docker).

2. Week 1-2 — Database + scraper

Design and create the products and price_history tables. Write raw SQL first to understand relationships, then wire up SQLAlchemy/SQLModel models. Build a simple scraper function for one test site or a mock store you control.

3. Week 2 — Core API endpoints

Build POST /products, GET /products, GET /products/{id}/history, GET /products/{id}/summary. Test everything using FastAPI's auto-generated /docs (Swagger UI) before touching the frontend.

4. Week 2-3 — Scheduling + alert logic

Add APScheduler to run the scraper on a timer, inserting new price_history rows. Add logic to compare current price against target_price and flag/alert when it drops.

5. Week 3 — Authentication

Add JWT login/signup endpoints. Protect the /products routes so each user only sees their own tracked items. Also a good week to write a few tests for your endpoints.

6. Week 3-4 — Frontend dashboard

Build a simple HTML page: a form to add a product, a list/table of tracked products with current price and % change, and a line chart (Chart.js) per product. Connect it to your API using JS `fetch()`.

7. Week 4 — Deploy + document

Deploy on Render or Railway with a managed Postgres add-on, confirm the scheduler runs in production. Write a clear README (problem it solves, schema diagram, how to run it) and push a clean commit history to GitHub.

Note: this timeline assumes roughly 1–1.5 hours/day. With more daily time available, it can reasonably be compressed to 2–3 weeks.